

École Marocaine des Sciences de l'Ingénieur (EMSI)

Filière 3IIR — Ingénierie Informatique et Réseaux



Rapport de stage

Conception et réalisation d'une application web de gestion des ventes de blocs de glace

« IceVentes » : plateforme Django pour l'Office National des Pêches

Réalisé par

Walid El Kiyesse

Encadré par

Abdelghani OUGRI

Structure d'accueil

Office National des Pêches (ONP)

Durée du stage : 1 mois (06 juillet – 06 août 2026)

Année académique 2025–2026

Résumé

Ce rapport présente la conception et la réalisation d'**IceVentes**, une application web de gestion des ventes de blocs de glace développée au cours d'un stage effectué au sein de l'Office National des Pêches (ONP). Dans le secteur de la pêche, la glace en blocs est indispensable à la conservation du poisson ; sa vente aux professionnels était jusqu'alors suivie de manière manuelle, sans centralisation ni traçabilité. Le projet vise à informatiser ce processus au moyen d'une application web moderne, sécurisée et simple d'utilisation.

La solution repose sur le cadre Python **Django**, une interface responsive construite avec Bootstrap 5 et HTMX, une base de données relationnelle (SQLite en développement, PostgreSQL en production) et des exports bureautiques (Excel, PDF). Elle couvre l'ensemble du cycle métier : authentification et gestion des utilisateurs avec trois rôles (Administrateur, Agent, Caissier) ; paramétrage d'un référentiel d'usines à glace (code, ville, prix unitaire en DH/Kg) ; gestion des demandeurs (clients, personnes physiques ou morales, acheteurs ou vendeurs) ; saisie d'une vente avec génération automatique d'un code unique BG_<réf>/<année> <numéro> et calcul automatique de la quantité ou du prix total ; enfin, un historique multicritère avec exports et un tableau de bord synthétisant l'activité (nombre de ventes, quantité vendue, chiffre d'affaires, demandeurs actifs) ainsi qu'un graphique d'évolution mensuelle.

Une attention particulière a été portée à la cohérence des données (unicité des codes, référentiel actif unique, numérotation fiable en concurrence), à la sécurité des accès (contrôle par rôle, protection contre les attaques par force brute) et à la qualité de l'expérience utilisateur. Le rapport détaille le cadre du projet, la spécification des besoins, la conception du système et sa réalisation effective, illustrée par les interfaces produites.

Mots clés : IceVentes, application web, Django, Python, gestion des ventes, blocs de glace, ONP, Bootstrap, HTMX, rôles, tableau de bord

Abstract

This report presents the design and implementation of **IceVentes**, a web application for managing ice-block sales, developed during an internship at the National Fisheries Office (Office National des Pêches, ONP). In the fishing sector, block ice is essential for preserving fish ; its sale to professionals was previously tracked manually, without centralisation or traceability. The project aims to digitise this process through a modern, secure and easy-to-use web application.

The solution is built on the Python **Django** framework, a responsive interface using Bootstrap 5 and HTMX, a relational database (SQLite in development, PostgreSQL in production) and office-document exports (Excel, PDF). It covers the whole business cycle : authentication and user management with three roles (Administrator, Agent, Cashier) ; configuration of an ice-factory reference (code, city, unit price in MAD/Kg) ; management of requesters (customers, natural or legal persons, buyers or sellers) ; sale entry with automatic generation of a unique code BG_<ref>/<year> <number> and automatic computation of quantity or total price ; and finally a multi-criteria sales history with exports and a dashboard summarising activity (number of sales, quantity sold, revenue, active requesters) together with a monthly trend chart.

Particular care was taken regarding data consistency (unique codes, a single active reference, reliable numbering under concurrency), access security (role-based control, brute-force protection) and user-experience quality. The report describes the project context, the requirements specification, the system design and its actual implementation, illustrated by the produced interfaces.

Keywords: IceVentes, web application, Django, Python, sales management, ice blocks, ONP, Bootstrap, HTMX, roles, dashboard

Table des matières

Résumé	i
Abstract	ii
Table des matières	iii
Liste des figures	v
Liste des tableaux	vi
Liste des abréviations	vii
Introduction générale	1
1 Présentation du cadre du projet	2
1.1 Présentation de l'organisme d'accueil	2
1.2 Contexte du projet	2
1.3 Problématique	3
1.4 Objectifs du projet	3
1.5 Conduite et planification du projet	3
1.6 Conclusion	4
2 Spécification des besoins	5
2.1 Identification des acteurs	5
2.2 Besoins fonctionnels	5
2.3 Besoins non fonctionnels	6
2.4 Diagramme de cas d'utilisation	6
2.5 Conclusion	8
3 Conception du système	9

3.1	Architecture logicielle	9
3.2	Conception de la base de données	9
3.3	Diagrammes de séquence	10
3.3.1	Authentification	10
3.3.2	Enregistrement d'une vente	11
3.3.3	Consultation de l'historique	12
3.4	Règles métier clés	12
3.5	Conclusion	12
4	Réalisation du système	14
4.1	Environnement et outils de développement	14
4.2	Présentation des interfaces	14
4.2.1	Tableau de bord	15
4.2.2	Gestion des utilisateurs	15
4.2.3	Référentiel	16
4.2.4	Gestion des demandeurs	17
4.2.5	Saisie d'une nouvelle vente	17
4.2.6	Historique des ventes	18
4.3	Focus sur une fonctionnalité clé : la génération du code de vente	19
4.4	Sécurité	19
4.5	Tests et validation	20
4.6	Conclusion	20
	Conclusion générale	21
	Bibliographie et Nétographie	22

Table des figures

1.1	Diagramme de Gantt : planification du stage	4
2.1	Diagramme de cas d'utilisation d'IceVentes	7
3.1	Diagramme de classes d'IceVentes	10
3.2	Diagramme de séquence : connexion d'un utilisateur	11
3.3	Diagramme de séquence : enregistrement d'une vente	11
3.4	Diagramme de séquence : consultation et filtrage de l'historique	12
4.1	Tableau de bord : indicateurs et évolution du chiffre d'affaires	15
4.2	Gestion des utilisateurs et des rôles	16
4.3	Référentiel : usine à glace et prix unitaire	16
4.4	Gestion des demandeurs (clients)	17
4.5	Saisie d'une nouvelle vente et génération du code BG	18
4.6	Historique des ventes : filtres multicritères et exports	18

Liste des tableaux

2.1	Acteurs du système et leurs responsabilités	5
2.2	Matrice des droits par rôle	7
4.1	Environnement technique et outils utilisés	14
4.2	Synthèse des validations réalisées	20

Liste des abréviations

Acronyme	Signification
API	Application Programming Interface
BG	Bloc de Glace
CIN	Carte d'Identité Nationale
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
CSV	Comma-Separated Values
DH	Dirham marocain
HTML	HyperText Markup Language
HTMX	Hypertext Markup eXtensions (bibliothèque JavaScript)
HTTP	HyperText Transfer Protocol
IHM	Interface Homme-Machine
KPI	Key Performance Indicator (indicateur clé de performance)
MVC	Model-View-Controller
MVT	Model-View-Template
ONP	Office National des Pêches
ORM	Object-Relational Mapping
PDF	Portable Document Format
PFE	Projet de Fin d'Études
SGBD	Système de Gestion de Base de Données
SQL	Structured Query Language
UI	User Interface
UML	Unified Modeling Language
URL	Uniform Resource Locator
UX	User Experience
WSGI	Web Server Gateway Interface

Introduction générale

La transformation numérique constitue aujourd’hui un levier majeur d’amélioration de la performance des organisations, y compris dans les établissements publics. En automatisant les tâches répétitives, en fiabilisant les données et en offrant une vision consolidée de l’activité, les applications de gestion permettent de gagner en efficacité, en traçabilité et en qualité de service.

C’est dans ce contexte que s’inscrit le présent stage, réalisé au sein de l’Office National des Pêches (ONP). L’ONP accompagne le développement du secteur de la pêche au Maroc et met notamment à disposition des professionnels de la glace en blocs, indispensable à la conservation du poisson depuis le débarquement jusqu’à la commercialisation. La vente de cette glace génère une activité récurrente qui, dans le cadre de ce projet, était suivie de façon essentiellement manuelle.

Cette gestion manuelle présente plusieurs limites : risque d’erreurs de saisie et de calcul, absence de numérotation fiable des tickets de vente, difficulté à retrouver l’historique des transactions, et impossibilité de disposer d’indicateurs synthétiques sur l’activité. L’objectif du stage est donc de **concevoir et réaliser une application web** qui centralise et sécurise la gestion des ventes de blocs de glace, tout en restant simple à utiliser au quotidien par les agents et caissiers.

L’application développée, baptisée **IceVentes**, couvre la gestion des utilisateurs et de leurs rôles, le paramétrage du prix de la glace, la gestion des clients, la saisie des ventes avec génération automatique d’un code unique, ainsi que la consultation de l’historique et le pilotage de l’activité à travers un tableau de bord.

Ce rapport est organisé en quatre chapitres :

- le **premier chapitre** présente le cadre du projet : l’organisme d’accueil, le contexte, la problématique, les objectifs et la conduite du projet ;
- le **deuxième chapitre** est consacré à la spécification des besoins fonctionnels et non fonctionnels, ainsi qu’à l’identification des acteurs et des cas d’utilisation ;
- le **troisième chapitre** détaille la conception du système : architecture, modèle de données et diagrammes de séquence ;
- le **quatrième chapitre** décrit la réalisation : environnement technique, interfaces produites, fonctionnalités clés et validation.

Le rapport se termine par une conclusion générale qui dresse le bilan du travail réalisé et propose des perspectives d’évolution.

Chapitre 1

Présentation du cadre du projet

Ce chapitre situe le projet dans son contexte. Il présente l'organisme d'accueil, expose la problématique métier à l'origine du besoin, précise les objectifs visés et décrit la démarche adoptée pour conduire le projet.

1.1 Présentation de l'organisme d'accueil

L'**Office National des Pêches (ONP)** est un établissement public marocain placé sous la tutelle du ministère chargé de la pêche maritime. Créé en 1969, il a pour mission de contribuer au développement du secteur de la pêche, de valoriser les produits de la mer et d'organiser leur commercialisation dans des conditions de qualité, de transparence et de traçabilité [1].

Parmi ses missions, l'ONP assure notamment la gestion et l'exploitation des halles aux poissons, l'organisation de la première vente du poisson, l'aménagement des points de débarquement, ainsi que la mise à disposition des professionnels d'infrastructures et de services de soutien à l'activité de pêche. La **fabrication et la distribution de glace** en font partie : la glace en blocs est un intrant essentiel de la chaîne du froid, car elle permet de préserver la fraîcheur du poisson depuis sa capture jusqu'à sa vente.

L'ONP se caractérise par une forte dimension opérationnelle et une présence sur l'ensemble du littoral marocain. Cet environnement, marqué par des exigences de fiabilité et de rigueur dans le suivi des opérations, constitue un cadre pertinent pour un projet d'informatisation d'un processus de vente.

1.2 Contexte du projet

La glace produite par les usines de l'ONP est vendue aux professionnels du secteur (mareyeurs, coopératives, sociétés de distribution, etc.). Chaque vente correspond à une quantité de glace, exprimée en kilogrammes, facturée sur la base d'un **prix unitaire** fixé par usine (en dirhams par kilogramme). Le suivi de ces ventes doit permettre d'identifier le client, de tracer chaque transaction par un numéro unique et de conserver un historique fiable.

Dans la situation initiale, ce suivi reposait largement sur des supports manuels. Cette approche, bien que fonctionnelle, atteint rapidement ses limites lorsque le volume d'opérations augmente et que le besoin de reporting se fait sentir.

1.3 Problématique

La gestion manuelle des ventes de blocs de glace soulève plusieurs difficultés :

- **Fiabilité des calculs** : le calcul de la quantité à partir du prix (ou inversement) est effectué à la main, ce qui est source d'erreurs ;
- **Numérotation des ventes** : l'attribution d'un numéro unique et croissant à chaque ticket est difficile à garantir, surtout en cas de saisies simultanées ;
- **Traçabilité** : retrouver une vente passée, filtrer par période, par client ou par montant, devient fastidieux ;
- **Pilotage** : l'absence d'indicateurs consolidés (volume vendu, chiffre d'affaires) prive les responsables d'une vision synthétique de l'activité ;
- **Sécurité et rôles** : rien ne distingue formellement les personnes autorisées à créer une vente de celles qui ne doivent que la consulter.

La question centrale est donc la suivante : *comment concevoir une application simple, fiable et sécurisée permettant de centraliser la gestion des ventes de blocs de glace, d'en garantir la cohérence et d'en faciliter le suivi ?*

1.4 Objectifs du projet

Pour répondre à cette problématique, le projet fixe les objectifs suivants :

- centraliser la saisie et la consultation des ventes dans une application web unique ;
- générer automatiquement un **code de vente unique** par ticket, de la forme BG_<réf>/<année> <numéro> ;
- automatiser le calcul entre **prix total** et **quantité** à partir du prix unitaire du référentiel ;
- gérer les utilisateurs selon trois **rôles** distincts (Administrateur, Agent, Caissier) ;
- paramétrer un référentiel d'usines à glace et gérer les clients (demandeurs) ;
- offrir un **historique multicritère** avec exports (Excel, PDF, impression) ;
- fournir un **tableau de bord** d'indicateurs et un graphique d'évolution ;
- garantir la cohérence des données et la sécurité des accès.

1.5 Conduite et planification du projet

Le projet a été mené selon une démarche itérative et incrémentale : après une phase de découverte de l'existant et d'analyse des besoins, les fonctionnalités ont été développées par lots successifs, chaque lot étant testé et validé avant de passer au suivant. Cette approche

a permis d'obtenir rapidement une application fonctionnelle, puis de l'enrichir et de la fiabiliser au fil du temps (correctifs, audit, nouvelles fonctionnalités, documentation).

La figure 1.1 présente le planning prévisionnel du stage sous la forme d'un diagramme de Gantt.

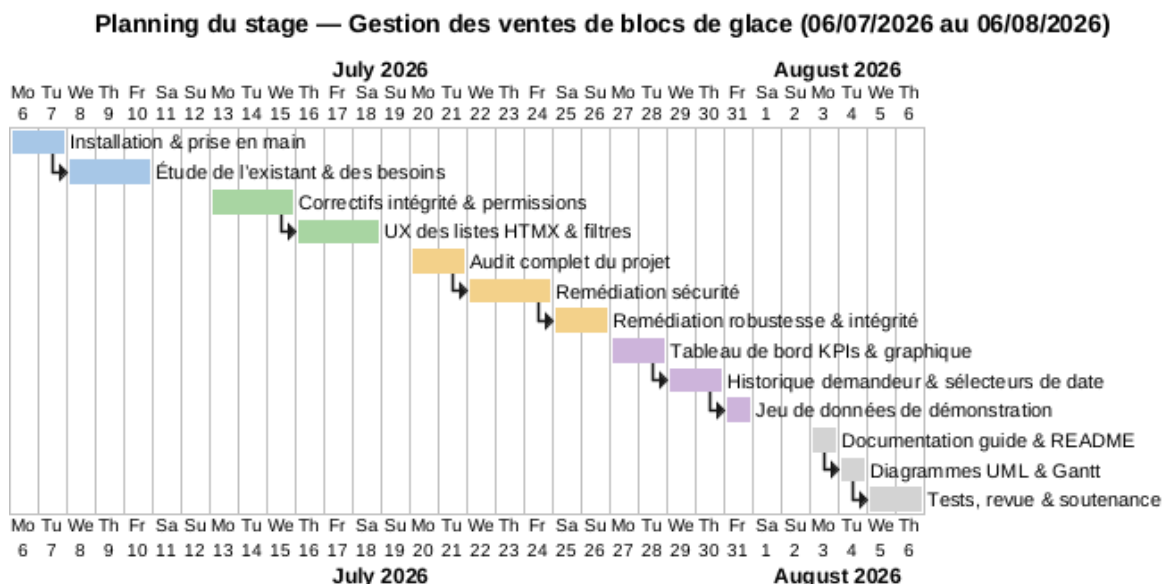


FIGURE 1.1 – Diagramme de Gantt : planification du stage

Le planning s'étend sur environ cinq semaines, réparties en cinq phases : découverte et analyse, correctifs et amélioration de l'expérience utilisateur, audit et remédiation, développement des fonctionnalités, puis documentation et finalisation. Chaque phase se termine par une validation avant le démarrage de la suivante.

1.6 Conclusion

Ce chapitre a présenté le cadre du projet : l'Office National des Pêches en tant que structure d'accueil, le contexte de la vente de glace, la problématique liée à sa gestion manuelle, les objectifs à atteindre et la démarche retenue. Le chapitre suivant traduit ces objectifs en une spécification détaillée des besoins.

Chapitre 2

Spécification des besoins

Ce chapitre formalise les besoins auxquels l'application doit répondre. Il identifie les acteurs du système, décrit les besoins fonctionnels et non fonctionnels, puis synthétise l'ensemble à travers un diagramme de cas d'utilisation.

2.1 Identification des acteurs

L'application distingue trois profils d'utilisateurs, correspondant à des responsabilités différentes au sein du processus de vente. Le tableau 2.1 récapitule ces acteurs et leur rôle.

TABLE 2.1 – Acteurs du système et leurs responsabilités

Acteur	Responsabilités
Administrateur	Accès complet : gestion des utilisateurs, du référentiel et des demandeurs, création et consultation des ventes, exports et pilotage.
Caissier	Création et consultation des ventes, impression des tickets et exports ; pas d'accès à l'administration.
Agent	Consultation seule : tableau de bord et historique des ventes, sans possibilité de création ni d'export.

2.2 Besoins fonctionnels

Les besoins fonctionnels sont organisés par module métier :

- **Authentification et utilisateurs** : connexion sécurisée ; gestion des comptes (création, modification, activation/désactivation) ; affectation d'un rôle ; recherche et filtrage des utilisateurs.
- **Référentiel** : paramétrage de l'usine à glace (code, nom, ville, prix unitaire en DH/Kg, logo) ; règle d'un **seul référentiel actif** à la fois, dont le prix alimente les ventes.
- **Demandeurs** : gestion des clients, personnes **physiques** (nom, prénom, CIN) ou **morales** (raison sociale, représentant) ; statut **acheteur** ou **vendeur** ; activa-

tion/désactivation ; recherche multicritère.

- **Ventes** : saisie d'une vente par sélection d'un demandeur et saisie du prix total **ou** de la quantité, l'autre valeur étant calculée automatiquement ; génération automatique du code BG_<réf>/<année> <numéro> ; impression du ticket.
- **Historique et pilotage** : consultation des ventes avec filtres (dates, demandeur, type, code, montant) ; exports Excel et PDF, impression ; tableau de bord avec indicateurs et graphique d'évolution.

2.3 Besoins non fonctionnels

Au-delà des fonctionnalités, l'application doit satisfaire un ensemble d'exigences de qualité :

- **Sécurité** : contrôle d'accès par rôle sur chaque écran, mots de passe hachés, protection contre les tentatives répétées de connexion (verrouillage) ;
- **Fiabilité et cohérence** : unicité des codes de vente, numérotation correcte même en cas d'accès concurrents, unicité du référentiel actif ;
- **Ergonomie** : interface claire et responsive, navigation intuitive, retours visuels et messages explicites ;
- **Performance** : pagination des listes et réponses rapides sur les écrans de consultation ;
- **Portabilité** : fonctionnement sur un simple poste (base SQLite) comme sur un serveur (PostgreSQL) ;
- **Maintenabilité** : code structuré, séparation claire des responsabilités et documentation.

2.4 Diagramme de cas d'utilisation

La figure 2.1 synthétise les interactions entre les acteurs et les fonctionnalités du système.

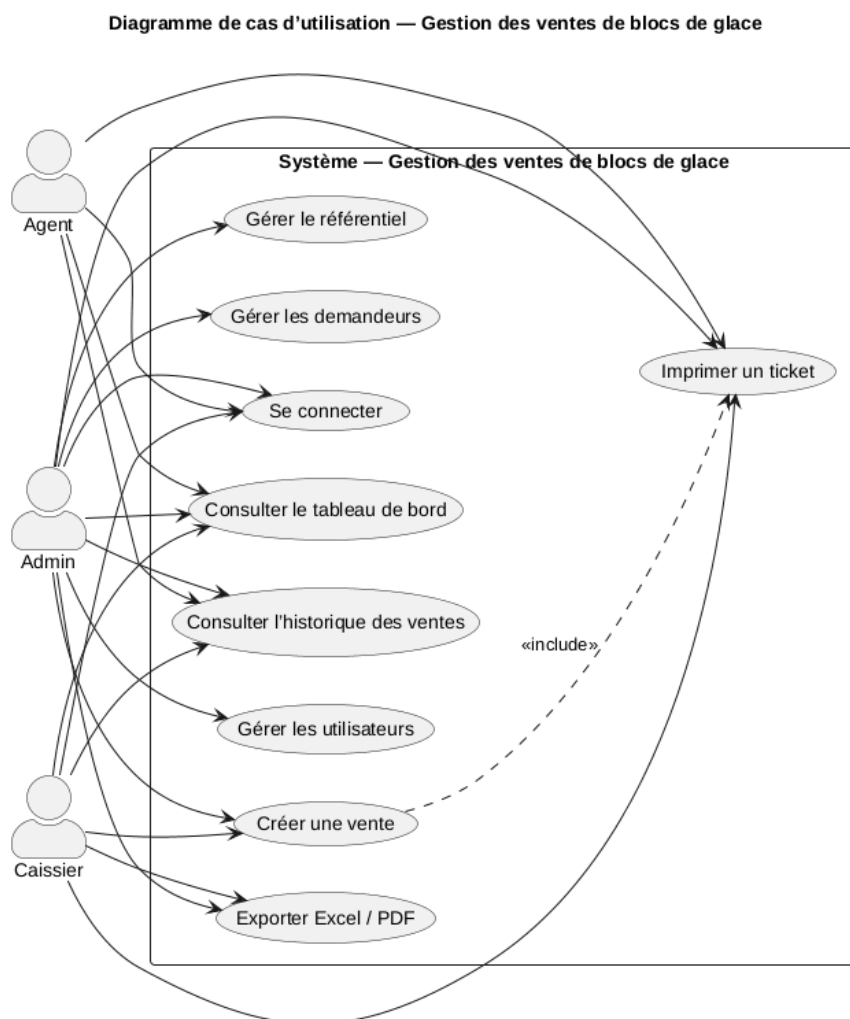


FIGURE 2.1 – Diagramme de cas d'utilisation d'IceVentes

Le diagramme met en évidence la séparation des droits : l'Agent est limité à la consultation (tableau de bord, historique, impression) ; le Caissier peut en plus créer des ventes et exporter ; l'Administrateur dispose de l'intégralité des fonctions, dont la gestion des utilisateurs, du référentiel et des demandeurs. La création d'une vente inclut l'impression d'un ticket.

Le tableau 2.2 précise la matrice des droits par rôle.

TABLE 2.2 – Matrice des droits par rôle

Fonctionnalité	Admin	Caissier	Agent
Se connecter	Oui	Oui	Oui
Tableau de bord	Oui	Oui	Oui
Historique des ventes	Oui	Oui	Oui
Créer une vente	Oui	Oui	Non
Exporter (Excel / PDF)	Oui	Oui	Non
Gérer les utilisateurs	Oui	Non	Non
Gérer le référentiel	Oui	Non	Non
Gérer les demandeurs	Oui	Non	Non

2.5 Conclusion

Ce chapitre a défini les besoins fonctionnels et non fonctionnels de l'application, ainsi que les acteurs et leurs droits. Ces éléments constituent le socle de la conception présentée dans le chapitre suivant.

Chapitre 3

Conception du système

Ce chapitre décrit la conception de l'application : son architecture logicielle, son modèle de données et la dynamique de ses principaux scénarios, illustrée par des diagrammes de séquence. Les règles métier essentielles sont également précisées.

3.1 Architecture logicielle

L'application est bâtie sur le cadriciel **Django**, qui repose sur le patron d'architecture **MVT** (Model-View-Template), variante du modèle MVC :

- le **Model** décrit les données et la logique métier ; il est projeté sur la base de données par l'ORM de Django ;
- la **View** traite les requêtes, applique les règles d'accès et prépare les données ;
- le **Template** produit les pages HTML envoyées au navigateur.

L'application suit une architecture **client-serveur** classique : le navigateur envoie des requêtes HTTP au serveur Django, qui interroge la base de données puis renvoie les pages. Côté interface, la bibliothèque **HTMX** permet d'actualiser certaines portions de page (listes filtrées, pagination, fenêtres de détail) sans recharger l'écran complet, ce qui fluidifie l'expérience tout en conservant une logique centralisée côté serveur.

Le code est organisé en modules fonctionnels (*applications* Django) : **accounts** (utilisateurs et authentification), **referentiel**, **demandeurs**, **ventes** et **core** (tableau de bord et éléments partagés). Cette séparation favorise la lisibilité et la maintenabilité.

3.2 Conception de la base de données

Le modèle de données est présenté par le diagramme de classes de la figure 3.1.

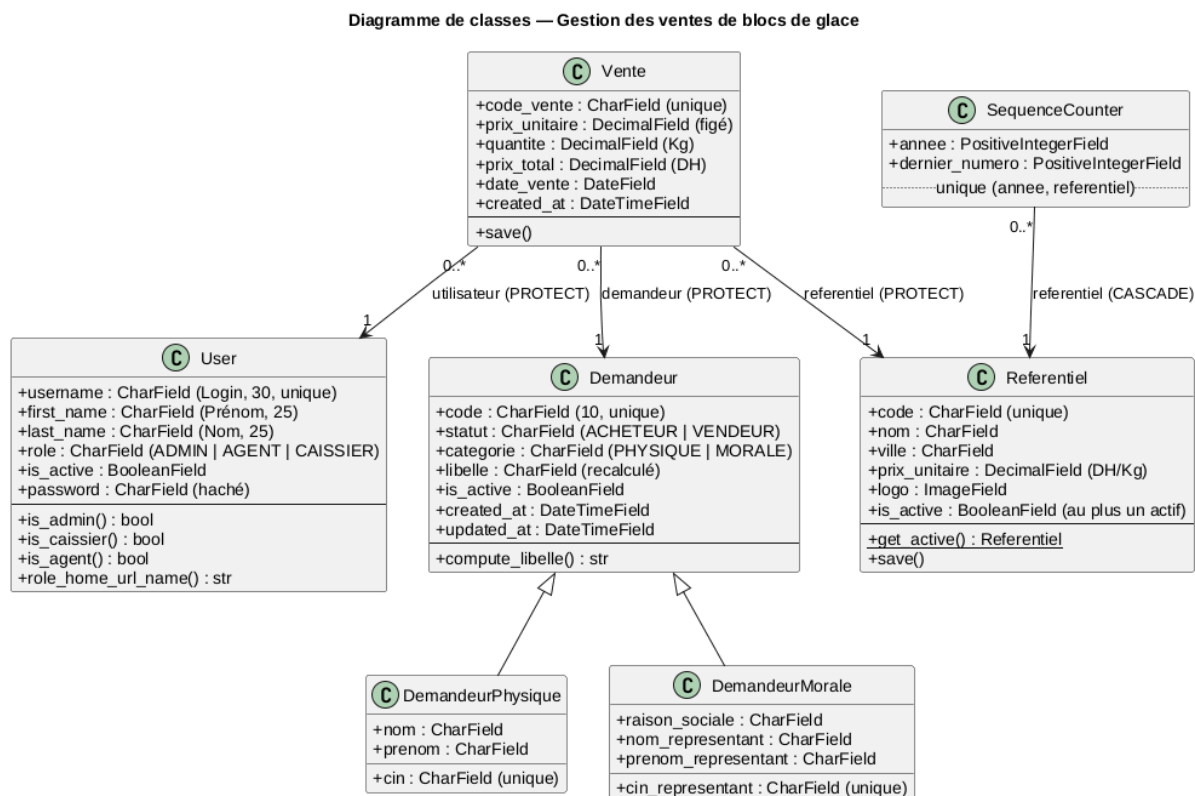


FIGURE 3.1 – Diagramme de classes d'IceVentes

Le diagramme regroupe les entités principales : **User** (utilisateur avec rôle), **Demandeur** et ses deux spécialisations **DemandeurPhysique** et **DemandeurMorale** (héritage), **Referentiel**, **Vente** et **SequenceCounter** (compteur de numérotation). Une vente référence un demandeur, un référentiel et l'utilisateur qui l'a saisie ; ces liens sont protégés (suppression interdite) afin de préserver l'intégrité de l'historique.

Le modèle appelle plusieurs remarques :

- les demandeurs sont modélisés par **héritage** : une classe de base **Demandeur** porte les attributs communs (code, statut, catégorie), tandis que **DemandeurPhysique** et **DemandeurMorale** ajoutent leurs champs spécifiques ;
- le **Referentiel** porte le **prix unitaire** qui sert de référence au calcul des ventes ; une contrainte garantit qu'un **seul référentiel** est actif à la fois ;
- la **Vente** conserve une copie (*figée*) du prix unitaire et du référentiel au moment de la vente, afin que l'historique reste exact même si le prix évolue par la suite ;
- le **SequenceCounter** assure une numérotation annuelle par référentiel, base de la génération du code de vente.

3.3 Diagrammes de séquence

3.3.1 Authentification

La figure 3.2 décrit le scénario de connexion, incluant la protection contre les tentatives répétées.

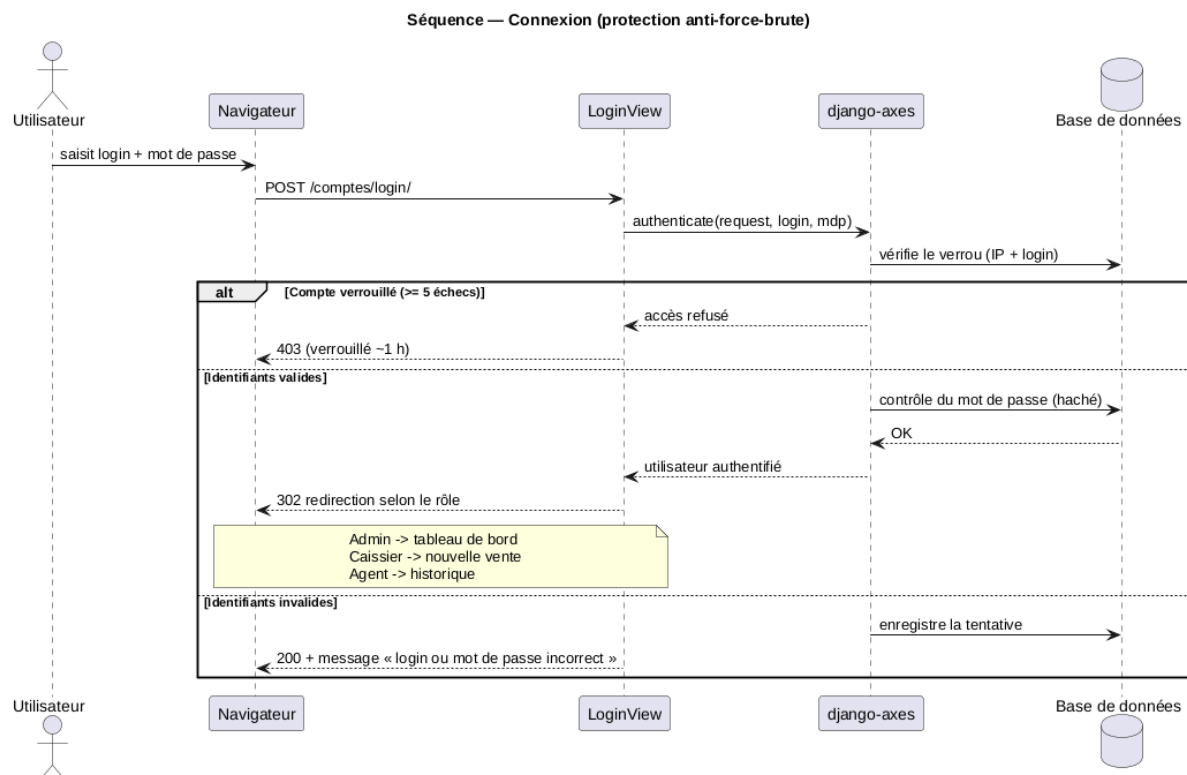


FIGURE 3.2 – Diagramme de séquence : connexion d'un utilisateur

L'utilisateur soumet ses identifiants ; le système vérifie d'abord qu'il n'est pas verrouillé (après plusieurs échecs), puis contrôle le mot de passe. En cas de succès, l'utilisateur est redirigé vers l'écran correspondant à son rôle ; en cas d'échec, la tentative est enregistrée et un message d'erreur est affiché.

3.3.2 Enregistrement d'une vente

La figure 3.3 illustre le scénario central de l'application : la saisie d'une vente et la génération de son code.

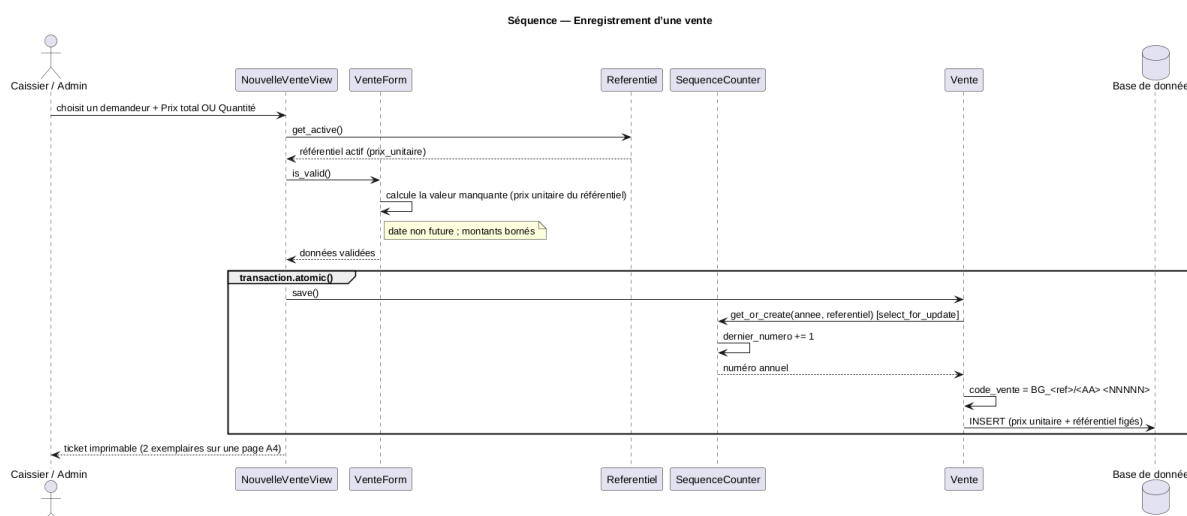


FIGURE 3.3 – Diagramme de séquence : enregistrement d'une vente

Après sélection du demandeur et saisie du prix total ou de la quantité, le système récupère le référentiel actif, calcule la valeur manquante, puis enregistre la vente dans une transaction : le compteur est incrémenté de manière verrouillée pour garantir un numéro unique, et le code BG définitif est attribué avant l'insertion en base. Un ticket imprimable est finalement proposé.

3.3.3 Consultation de l'historique

La figure 3.4 présente le filtrage de l'historique, réalisé sans rechargement complet grâce à HTMX.

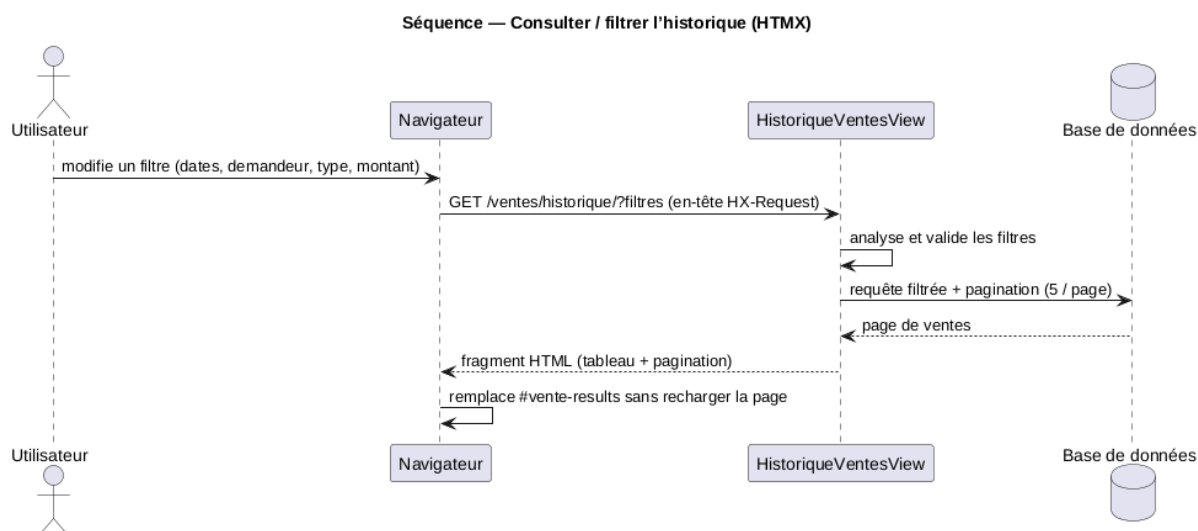


FIGURE 3.4 – Diagramme de séquence : consultation et filtrage de l'historique

Lorsque l'utilisateur modifie un filtre, une requête est envoyée au serveur, qui interroge la base avec les critères et la pagination, puis renvoie uniquement le fragment de tableau mis à jour. Le navigateur remplace la portion concernée de la page, ce qui rend l'interaction fluide.

3.4 Règles métier clés

Trois règles structurantes garantissent la cohérence du système :

- **Code de vente unique** : chaque vente reçoit un code BG_<réf>/<année> <numéro>, où le numéro est croissant par année et par référentiel ; son attribution est protégée en concurrence par une transaction et un verrou ;
- **Référentiel actif unique** : au plus un référentiel peut être actif ; c'est son prix unitaire qui alimente toutes les nouvelles ventes ;
- **Cohérence des demandeurs** : le code d'un demandeur est unique, et le CIN est contrôlé de façon à éviter les doublons entre personnes physiques et morales.

3.5 Conclusion

Ce chapitre a exposé l'architecture MVT retenue, le modèle de données et les principaux scénarios dynamiques de l'application, ainsi que les règles métier garantissant sa cohérence.

Le chapitre suivant présente la concrétisation de cette conception.

Chapitre 4

Réalisation du système

Ce chapitre présente la réalisation effective de l'application : l'environnement et les outils utilisés, les interfaces produites, une fonctionnalité clé illustrée par un extrait de code, les aspects de sécurité et enfin la validation.

4.1 Environnement et outils de développement

L'application a été développée avec un ensemble d'outils libres et éprouvés, récapitulés dans le tableau 4.1.

TABLE 4.1 – Environnement technique et outils utilisés

Outil / Technologie	Rôle dans le projet
Python	Langage de programmation principal.
Django	Cadriciel web (ORM, routage, gabarits, administration).
SQLite / PostgreSQL	Base de données (développement / production).
HTML, CSS, Bootstrap 5	Structure et mise en forme responsive des pages.
HTMX	Mises à jour partielles de page (filtres, pagination, modales).
Chart.js	Graphique d'évolution du chiffre d'affaires.
django-crispy-forms	Rendu soigné des formulaires.
openpyxl	Génération des exports Excel.
xhtml2pdf	Génération des exports PDF.
django-axes	Protection contre les attaques par force brute.
Flatpickr	Sélecteur de date au format jj/mm/aaaa.
UML / PlantUML	Modélisation et génération des diagrammes.
Git	Gestion de versions du code source.

4.2 Présentation des interfaces

Cette section illustre les principaux écrans de l'application.

4.2.1 Tableau de bord

Le tableau de bord (figure 4.1) constitue la page d'accueil ; il offre une vision synthétique de l'activité.

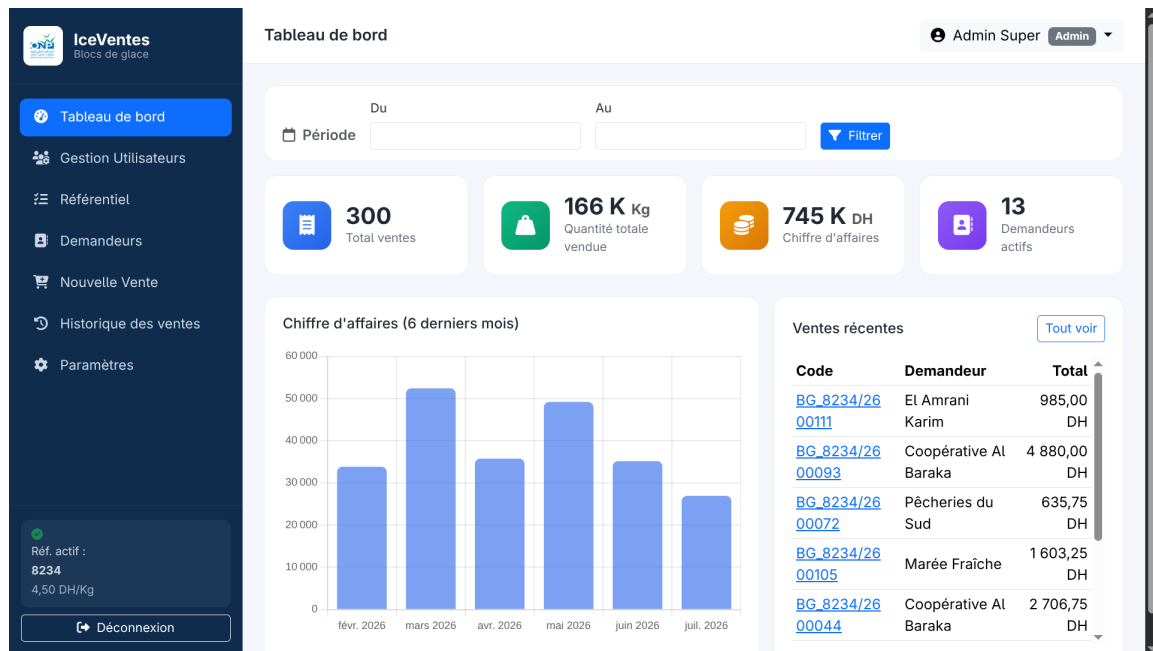


FIGURE 4.1 – Tableau de bord : indicateurs et évolution du chiffre d'affaires

Le tableau de bord affiche quatre indicateurs clés (nombre total de ventes, quantité vendue en Kg, chiffre d'affaires en DH et nombre de demandeurs actifs), présentés en notation compacte (K, M). Un graphique donne l'évolution du chiffre d'affaires sur les six derniers mois, et un panneau liste les ventes récentes. Un filtre par période permet de restreindre les statistiques à une plage de dates.

4.2.2 Gestion des utilisateurs

L'écran de gestion des utilisateurs (figure 4.2) est réservé à l'administrateur.

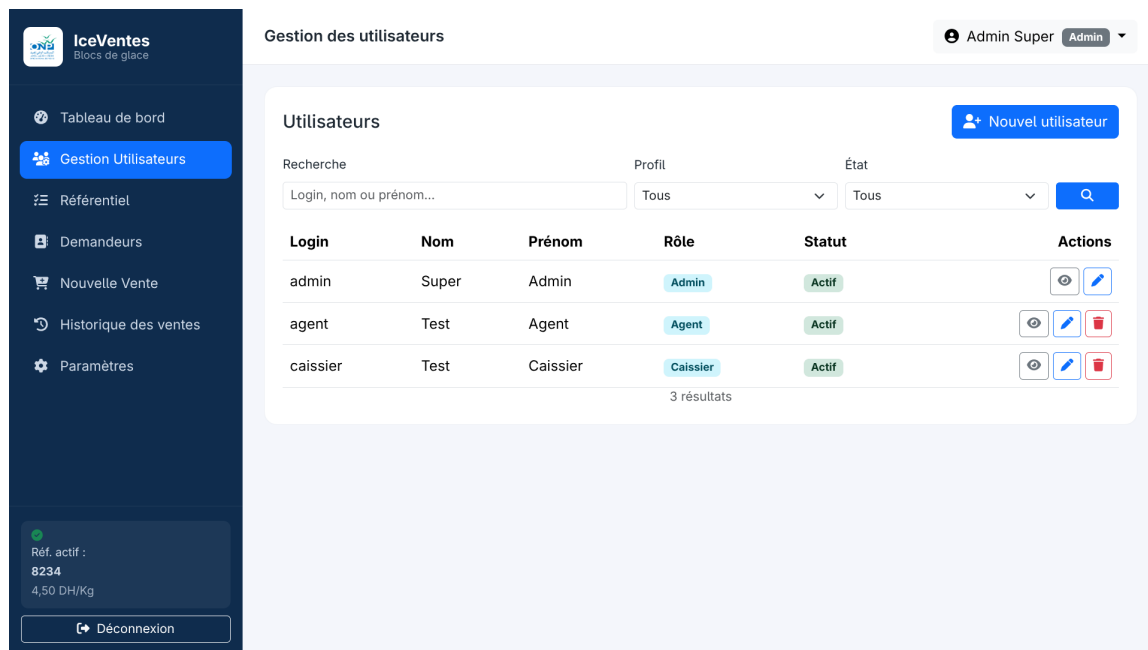


FIGURE 4.2 – Gestion des utilisateurs et des rôles

La liste des utilisateurs affiche le login, le nom, le prénom, le rôle (Admin, Agent, Caissier) et le statut (actif ou désactivé). Une barre de recherche et des filtres par profil et par état facilitent la navigation. Les actions permettent de consulter, modifier et supprimer un compte, ou d'en créer un nouveau.

4.2.3 Référentiel

Le référentiel (figure 4.3) paramètre l'usine à glace et son prix.

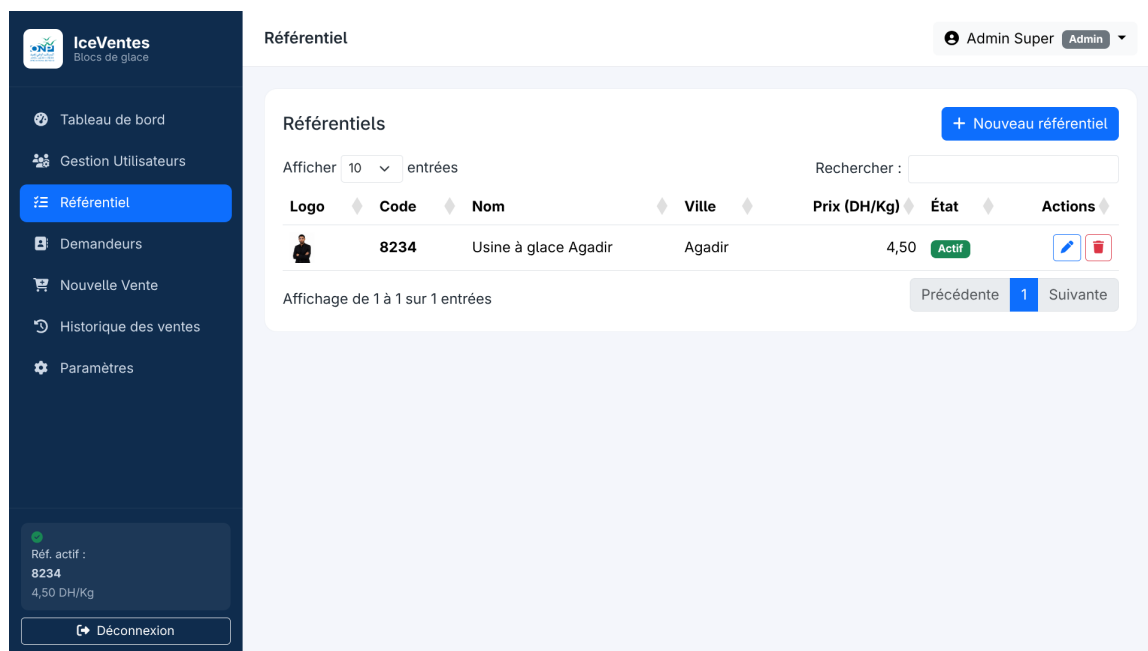


FIGURE 4.3 – Référentiel : usine à glace et prix unitaire

Le référentiel décrit l'usine à glace par un code, un nom, une ville, un logo et un prix unitaire en DH/Kg. Un seul référentiel est actif à la fois : c'est son prix (ici 4,50 DH/Kg pour l'usine d'Agadir) qui alimente automatiquement le calcul des nouvelles ventes.

4.2.4 Gestion des demandeurs

La gestion des demandeurs (figure 4.4) recense les clients.

IceVentes
Blocs de glace

Tableau de bord
Gestion Utilisateurs
Référentiel
Demandeurs
Nouvelle Vente
Historique des ventes
Paramètres

Réf. actif :
8234
4,50 DH/Kg
Déconnexion

Gestion des demandeurs

Admin Super Admin

Demandeurs + Nouveau demandeur

Recherche: Code, CIN, nom, prénom, raison sociale... Type: Tous Statut: Tous État: Tous

Code	Libellé	Catégorie	Statut	État	Actions
SP-002	Benali Youssef	Physique	Acheteur	Actif	
SP-006	Bennani Karim	Physique	Vendeur	Actif	
SP-004	Bennani Omar	Physique	Vendeur	Actif	
SP-003	Chraibi Mohamed	Physique	Vendeur	Actif	
DM-0001	Coopérative Al Baraka	Morale	Vendeur	Actif	

« 1 2 3 »
13 résultats — page 1 / 3

FIGURE 4.4 – Gestion des demandeurs (clients)

Les demandeurs peuvent être des personnes physiques ou morales, avec un statut acheteur ou vendeur. La liste, paginée, propose une recherche (code, CIN, nom, raison sociale) et des filtres par type, statut et état. Chaque demandeur peut être consulté, activé/désactivé, modifié ou supprimé.

4.2.5 Saisie d'une nouvelle vente

L'écran de saisie (figure 4.5) est au cœur de l'application.

IceVentes
Blocs de glace

Nouvelle vente Admin Super Admin

Code BG: BG_8234/26 00120 Date de vente: 03/08/2026
Généré automatiquement à l'enregistrement.

Demandeur: SP-002 — Benali Youssef [Choisir]

Prix unitaire (DH/Kg): 4,50 Prix total (DH): 0.00 Quantité (Kg): 0.000

Saisissez le **prix total** ou la **quantité** : l'autre valeur se calcule automatiquement à partir du prix unitaire.

[Enregistrer] [Historique]

Réf. actif : 8234
4,50 DH/Kg
Déconnexion

FIGURE 4.5 – Saisie d’une nouvelle vente et génération du code BG

Le code BG est généré automatiquement à l’enregistrement (ici BG_8234/26 00120). Après avoir choisi un demandeur, l’utilisateur saisit soit le prix total, soit la quantité : l’autre valeur est calculée instantanément à partir du prix unitaire du référentiel. La date de vente et le prix unitaire sont pré-remplis.

4.2.6 Historique des ventes

L’historique (figure 4.6) permet la consultation et l’export.

IceVentes
Blocs de glace

Historique des ventes Admin Super Admin

Date début: Date fin: Demandeur: code ou nom Type: Tous Code BG: BG... Montant min: 0 Montant max: ∞

Résultats [Excel] [PDF] [Imprimer]

Code BG	Demandeur	Quantité (Kg)	Prix Unitaire	Prix Total	Date	Utilisateur	Actions
BG_8234/26 00111	El Amrani Karim	218,889	4,50	985,00	25/07/2026	Admin Super	[View] [Print]
BG_8234/26 00093	Coopérative Al Baraka	1 084,444	4,50	4 880,00	24/07/2026	Admin Super	[View] [Print]
BG_8234/26 00072	Pêcheries du Sud	141,278	4,50	635,75	22/07/2026	Admin Super	[View] [Print]
BG_8234/26 00105	Marée Fraîche	356,278	4,50	1 603,25	21/07/2026	Caissier Test	[View] [Print]
BG_8234/26 00044	Coopérative Al Baraka	601,500	4,50	2 706,75	21/07/2026	Admin Super	[View] [Print]

Réf. actif : 8234
4,50 DH/Kg
Déconnexion

FIGURE 4.6 – Historique des ventes : filtres multicritères et exports

L'historique présente les ventes avec leur code, demandeur, quantité, prix unitaire, prix total, date et utilisateur. Une barre de filtres (dates, demandeur, type, code, montant minimum et maximum) affine les résultats sans rechargement complet. Les boutons Excel, PDF et Imprimer permettent d'exporter la sélection.

4.3 Focus sur une fonctionnalité clé : la génération du code de vente

Le code de vente est produit par un service dédié, exécuté dans une transaction afin de garantir un numéro unique même en cas de saisies simultanées. Le compteur associé à l'année et au référentiel est verrouillé le temps de l'incrément (`select_for_update`). L'extrait suivant en présente le principe.

Listing 4.1 – Génération du code de vente (principe)

```
1 def generer_code_vente(referentiel, date_vente):
2     annee = date_vente.year
3     with transaction.atomic():
4         compteur, _ = SequenceCounter.objects \
5             .select_for_update() \
6             .get_or_create(annee=annee, referentiel=
7                 referentiel)
8         compteur.dernier_numero += 1
9         compteur.save(update_fields=["dernier_numero"])
10        numero = compteur.dernier_numero
11    aa = f"{annee % 100:02d}"
12    return f"BG_{referentiel.code}/{aa} {numero:05d}"
```

Le même principe de calcul *côté serveur* s'applique à la vente : la quantité et le prix total sont toujours recalculés à partir du prix unitaire du référentiel, la valeur saisie par l'utilisateur servant de point de départ. Cela évite toute incohérence et ne fait pas confiance aux valeurs éventuellement modifiées côté client.

4.4 Sécurité

Plusieurs mesures renforcent la sécurité de l'application :

- **Contrôle d'accès par rôle** : chaque écran vérifie que l'utilisateur dispose des droits requis ; l'Agent, par exemple, ne peut pas accéder à la création de ventes ni aux exports ;
- **Protection anti-force-brute** : après plusieurs échecs de connexion, l'accès est temporairement verrouillé (django-axes) ;
- **Mots de passe hachés** et validation des entrées à la frontière du système ;
- **Intégrité des données** : unicité des codes, référentiel actif unique et numérotation fiable garantis au niveau de la base et de la logique métier.

4.5 Tests et validation

La validation a combiné des vérifications automatiques et des tests manuels des parcours par rôle. Le tableau 4.2 en présente une synthèse.

TABLE 4.2 – Synthèse des validations réalisées

Élément validé	Méthode	Résultat
Contrôle d'accès par rôle	Tests automatisés	Conforme
Génération du code de vente	Tests unitaires	Codes uniques et croissants
Calcul prix / quantité	Tests unitaires	Cohérent avec le prix unitaire
Référentiel actif unique	Tests + contrainte base	Un seul actif garanti
Filtres de l'historique	Tests fonctionnels	Résultats corrects
Exports Excel / PDF	Vérification fonctionnelle	Fichiers générés
Verrouillage anti-force-brute	Tests automatisés	Verrouillage effectif

L'ensemble des tests automatisés s'exécute sans erreur, ce qui confirme la fiabilité des règles métier et des contrôles d'accès mis en place.

4.6 Conclusion

Ce chapitre a présenté la réalisation concrète d'IceVentes : son environnement technique, ses interfaces, une fonctionnalité clé, ses mécanismes de sécurité et sa validation. L'application couvre l'ensemble des objectifs fixés et fonctionne de bout en bout.

Conclusion générale

Le stage réalisé au sein de l'Office National des Pêches avait pour objectif de concevoir et de réaliser une application web de gestion des ventes de blocs de glace, en réponse à une gestion jusqu'alors manuelle, source d'erreurs et dépourvue de traçabilité.

L'application **IceVentes** atteint cet objectif. Elle centralise l'ensemble du processus : authentification et gestion des utilisateurs selon trois rôles, paramétrage du référentiel et de son prix, gestion des demandeurs, saisie des ventes avec génération automatique d'un code unique et calcul automatisé entre prix et quantité, historique multicritère avec exports, et tableau de bord de pilotage. Une attention constante a été portée à la cohérence des données, à la sécurité des accès et à la qualité de l'expérience utilisateur.

Sur le plan personnel, ce projet a été l'occasion de mettre en pratique et d'approfondir de nombreuses compétences : la conception et la modélisation UML, le développement web avec Django, la manipulation d'une base de données relationnelle, la construction d'interfaces responsives et interactives, ainsi que la prise en compte des exigences de sécurité et de fiabilité propres à une application de gestion. Le travail dans un cadre professionnel a également renforcé la rigueur, l'autonomie et la capacité à structurer un projet de bout en bout.

Plusieurs **perspectives** d'évolution peuvent être envisagées :

- la mise en production sur une base PostgreSQL et un serveur dédié ;
- la prise en charge de plusieurs usines à glace actives simultanément ;
- l'enrichissement du reporting (statistiques avancées, comparaisons, export planifié) ;
- l'ajout d'une gestion fine des paiements et des encaissements ;
- le développement d'une version mobile pour la saisie sur le terrain.

En définitive, IceVentes constitue une base solide, fonctionnelle et extensible, qui répond aux besoins exprimés tout en ouvrant la voie à des améliorations futures.

Bibliographie

- [1] Office National des Pêches (ONP). *Site officiel de l'Office National des Pêches*. <http://www.onp.co.ma/>. Consulté en août 2026.
- [2] Django Software Foundation. *Django Documentation*. <https://docs.djangoproject.com/>. Consulté en août 2026.
- [3] Python Software Foundation. *Python Language Documentation*. <https://docs.python.org/3/>. Consulté en août 2026.
- [4] Bootstrap. *Bootstrap 5 Documentation*. <https://getbootstrap.com/docs/5.3/>. Consulté en août 2026.
- [5] HTMX. *HTMX Documentation*. <https://htmx.org/docs/>. Consulté en août 2026.
- [6] Chart.js. *Chart.js Documentation*. <https://www.chartjs.org/docs/>. Consulté en août 2026.
- [7] django-axes. *django-axes Documentation*. <https://django-axes.readthedocs.io/>. Consulté en août 2026.
- [8] openpyxl. *openpyxl Documentation*. <https://openpyxl.readthedocs.io/>. Consulté en août 2026.
- [9] PostgreSQL Global Development Group. *PostgreSQL Documentation*. <https://www.postgresql.org/docs/>. Consulté en août 2026.
- [10] PlantUML. *PlantUML Language Reference Guide*. <https://plantuml.com/>. Consulté en août 2026.